AI Development Prompts & Stack — By Role

Companion to `minic-language-spec.md` and `project-workflow-and-action-plan.md`. Paste the relevant spec sections into Claude Code as context before running these prompts — it'll match the team's locked conventions instead of guessing.

Person A — Front-end + IR + feature extractor

Stack: Python, hand-written recursive-descent parser (no parser-generator needed for a grammar this size), `dataclasses` for AST/TAC node types, `pytest`.

Prompts, in order:

1. *"Build a Python lexer for this MiniC grammar [paste BNF from the spec]. Tokens: keywords (int, float, char, struct, if, else, while, for, return), identifiers, int/float/char/string literals, operators, punctuation. Use a Token dataclass*

with type, value, and line number. Add unit tests for each token category."

2. *"Using this lexer, build a recursive-descent parser matching this BNF [paste grammar] that produces an AST using dataclasses for each node type. Report line numbers on syntax errors."*
3. *"Add semantic analysis: nominal struct type matching, array dimension checking, rejecting invalid lvalues (e.g. a function call or literal on the left of `=`)."*
4. *"Lower the type-checked AST to three-address code using this opcode set [paste the team's agreed TAC format]. Handle array indexing and struct field access as address computation followed by load/store."*
5. *"Write a feature extractor that walks the AST/TAC and computes: loop count, max loop nesting depth, basic block count, branch density, instruction-type histogram, struct field count, array dimensionality. Output as a flat dict."*
6. *"Run this pipeline against the canonical example program from the spec and show me the resulting TAC so I can check it by hand."*

Person B — Optimization passes + codegen

Stack: Python, same AST/TAC dataclasses as A, `pytest` regression suite.

Prompts, in order:

1. *"Given this TAC format [paste A's opcode spec], implement constant folding as a standalone toggleable pass — evaluate compile-time-constant expressions and replace with literals."*
2. *"Implement dead code elimination: backward liveness analysis, remove instructions whose result is never subsequently used."*
3. *"Implement common subexpression elimination using value numbering per basic block."*
4. *"Implement loop-invariant code motion: detect natural loops via back-edges, hoist operand-invariant instructions to the loop preheader."*
5. *"Implement strength reduction: replace multiplication by a loop induction variable with repeated addition."*
6. *"Wire all 5 passes behind a single 5-flag config so any of the 64 combinations can be applied in a fixed canonical order."*
7. *"Write codegen that translates TAC to C, wrapping every array/struct/string in a one-field*

C struct per Section 4 of the language spec, so assignment and parameter passing stay copy-semantics in the emitted C."

8. *"Write a regression test that runs all 64 combinations against the canonical example and asserts every one compiles and runs to output 83 — catch any pass that silently breaks program semantics."*

Person C — Timing harness + ML model + demo

Stack: Python, `subprocess` + system `gcc`, `pandas`, `scikit-learn` (random forest or gradient boosting — the dataset is small, deep learning is unnecessary here), optional `streamlit` for the demo UI.

Prompts, in order:

1. *"Write a function that compiles a C file with `gcc -O0 -o out`, runs `out` 20 times via subprocess timing with `time.perf_counter()`, discards the first 3 runs as warm-up, and returns the median in milliseconds."*
2. *"Wrap that in a sweep function that runs it across all 64 optimization-flag combinations for a given MiniC program and returns a pandas DataFrame of (program_id, combo_bits, features, median_time_ms)."*

3. *"Write a fake-data generator producing random feature vectors and random timings in the same schema, so the training pipeline can be built before real data exists."*
4. *"Build a training script using scikit-learn. Try both a classification framing (predict on/off per pass) and a regression framing (predict speedup per combo, pick the argmax). Report cross-validated performance for both and tell me which generalizes better to held-out programs."*
5. *"Build a small demo: given a MiniC program, extract features, predict the recommended combo, compile and run both the baseline and the recommended version, and print the measured speedup as a percentage."*

Person D — Benchmark corpus + correctness oracle + report section

Stack: Plain MiniC/C syntax written by hand — his actual strength — plus real `gcc` for ground truth. Claude Code writes any supporting automation so he never has to touch Python himself.

Prompts, in order:

1. *"Write 8-10 MiniC test programs, each exercising one specific feature with real computational weight (loop trip counts in the*

thousands+, not just correctness-checks): 2D array traversal, struct-heavy arithmetic, fixed-string comparison, deep recursion, an LICM-friendly loop, a strength-reduction-friendly loop, a mixed-feature program, and a large loop purely for timing headroom."

2. *"For each program, also give me the equivalent plain C (not MiniC-specific) so I can compile it with real gcc and use the output as ground truth."*
3. *"Write a short script that compiles all the reference C versions with gcc, runs them, and outputs a table of (program_name, expected_output) I can diff our own compiler's results against later."*
4. *"Draft a first pass of the report section explaining why MiniC excludes pointers and dynamic memory, and why arrays/structs use value semantics — written for a compiler-design audience, referencing alias analysis as the complexity this avoids."* (He edits and philosophizes over this draft rather than starting from a blank page.)